

ASSIGNMENT 2

NAME: SIBGHATULLAH

ROLL NO: 24P-0039

Q1:

```
import json
import requests
import pandas as pd

try:
    with open("api.txt", "r") as f:
        api_url = f.read().strip()
        if not api_url:
            print("Error: Please can you add the url into the txt file")
            exit()
except FileNotFoundError:
    print("Error: file not found, please upload the file")
    exit()

try:
    response = requests.get(api_url, timeout=10)
    if response.status_code != 200:
        print(f"HTTP Error: {response.status_code}")
        print("Response:", response.text)
    else:
        data = response.json()
        print("data fetched successfully")
        print(data)
except requests.exceptions.RequestException:
    print("Network Error: Please check your internet connection")

if data.get("status") != "success":
    reason = data.get("reason", "Unknown error")
    print(f"API Error: {reason}")
    exit()

matches = data.get("data", [])
if not matches:
    print("No live/current matches found.")
    exit()

for match in matches:
    print("="*70)
    print(f"Match Name -> {match.get('name', 'N/A')}")
    print(f>Status    -> {match.get('status', 'N/A')}")
    print(f>Venue     -> {match.get('venue', 'N/A')}")
    print(f>Date      -> {match.get('date', 'N/A')}")

    scores = match.get("score", [])
    if scores:
```

```

print("Scores:")
for inning in scores:
    print(f" Inning => {inning.get('inning', 'N/A')}")
    print(f" Runs  => {inning.get('r', 'N/A')}")
    print(f" Wickets=> {inning.get('w', 'N/A')}")
    print(f" Overs => {inning.get('o', 'N/A')}")
    print("="*70)
print("----"*15)

```

OUTPUT:

```

if not matches:
    print("No live/current matches found.")
    exit()

data fetched successfully
{'apikey': '818eadc7-396c-40e7-81e1-2105612be6b9', 'data': [{'id': '8d51f6fa-0cfb-4339-aacc-ad76429add78', 'name': 'Thailand Women vs

```

```

=====
Match Name -> Thailand Women vs Namibia Women, 2nd Match
Status      -> Thailand Women won by 39 runs
Venue       -> Terdthai Cricket Ground, Bangkok
Date        -> 2025-11-14
=====
Match Name -> Scotland Women vs Papua New Guinea Women, 1st Match
Status      -> Scotland Women won by 55 runs
Venue       -> Terdthai Cricket Ground, Bangkok
Date        -> 2025-11-14
=====
Match Name -> Pakistan vs Sri Lanka, 2nd ODI
Status      -> Pakistan need 206 runs
Venue       -> Rawalpindi Cricket Stadium, Rawalpindi
Date        -> 2025-11-13
=====
Match Name -> Western Australia vs Queensland, 12th Match
Status      -> Western Australia won by 1 wkt
Venue       -> W.A.C.A. Ground, Perth
Date        -> 2025-11-11
=====
Match Name -> India A vs South Africa A, 1st Unofficial ODI
Status      -> India A won by 4 wkts
Venue       -> Niranjan Shah Stadium, Khandheri, Rajkot
Date        -> 2025-11-13
=====

```

```

Scores:
Inning => south africa Inning 1
Runs   => 159
Wickets=> 10
Overs  => 55
=====
Inning => India,South Africa Inning 1
Runs   => 37
Wickets=> 1
Overs  => 20
=====
-----

```

Q2:

```

import numpy as np
fi = []
with open('sensor_data.csv', 'r') as f:
    for i in f:

```

```

        data = i.strip().split(',')
        fi.append([float(x) for x in data])
    n_array = np.array(fi)
    n_array
n_array[n_array == -999] = np.nan
n_array[(n_array < 0) & (n_array > 100)] = np.nan
n_array
ms = np.nanmean(n_array,axis=0) # for columns we use axis = 0
print("mean moisture for each sensor is: ",ms)
mes = np.nanmedian(n_array, axis = 1)
print("the meidan moisture for each hour is: ",mes)
nan_count=np.sum(np.isnan(n_array) , axis = 0)
nan_count.shape
np.argmax(nan_count)
m = np.nanmax(n_array)
n = np.nanmin(n_array)
print("maximum reading is: ",m,"\n","minimum reading is: ",n)
x_norm = (n_array - n)/(m - n)
x_norm[:5]
np.savetxt('sensor_data_normalized.csv',x_norm, delimiter=',', fmt='%.4f')

```

OUTPUT:

```

n_array
array([[ 21.21,  66.86,  47.73, ...,  60.63,  10.51,  83.58],
       [ 47.24,  71.7 ,  68.44, ...,  68.91,  29.68,  38.72],
       [ 39.07,  21.85,  31.86, ...,  12.77,  12.38,  16.5 ],
       ...,
       [ 18.6 ,  64.89,  41.9 , ...,  16.49,  32.04,  36.97],
       [ 22.03,  43.26,  55.61, ...,  13.56,  83.69,  61.68],
       [ 45.06, -999. ,  20.37, ...,  86.32,  35.03,  34.74]],
      shape=(720, 100))

```

```

n_array[(n_array < 0) & (n_array > 100)] = np.nan

n_array
array([[21.21, 66.86, 47.73, ..., 60.63, 10.51, 83.58],
       [47.24, 71.7 , 68.44, ..., 68.91, 29.68, 38.72],
       [39.07, 21.85, 31.86, ..., 12.77, 12.38, 16.5 ],
       ...,
       [18.6 , 64.89, 41.9 , ..., 16.49, 32.04, 36.97],
       [22.03, 43.26, 55.61, ..., 13.56, 83.69, 61.68],
       [45.06, nan, 20.37, ..., 86.32, 35.03, 34.74]], shape=(720, 100))

```

```
print("mean moisture for each sensor is: ",ms)
```

Pyth

```
mean moisture for each sensor is: [50.73858198 50.2341159 51.27293413 49.0773494 50.26127246 49.66935725
51.52051051 50.68205621 51.33179878 51.47947526 50.84037764 51.0440625
49.26940916 51.26822653 49.37944193 49.71962798 49.21847988 49.3203139
50.15004431 52.84455213 50.17457903 48.00882963 49.75005952 50.404003
49.19879056 50.36595238 50.2916716 49.01795588 49.63189552 50.99672083
50.95953731 49.36990991 50.82856509 50.03372372 51.63781627 51.60602679
48.96136499 50.19457652 49.54323308 49.5746027 50.55213115 50.19353372
50.19954876 49.76128743 51.57152207 49.78535127 51.35110783 48.45413534
50.27472059 48.89479104 49.9655988 51.89755224 49.3852608 49.93882006
49.58385757 51.64714925 48.61207849 50.08163959 51.58085294 48.52678042
50.17258443 51.64890045 50.21910045 51.03953453 50.323161 49.6522571
50.5086931 50.89558174 48.87628319 50.056997 48.0095188 48.66048817
50.25579882 50.08786765 48.84652632 49.44060029 50.80411411 50.8589497
49.87075668 52.08046547 52.45081241 50.79524812 49.91267936 50.91968182
49.73757801 50.18615385 48.47777943 49.48171554 47.30899701 49.69410104
48.62253333 50.6516568 49.0706278 51.51571006 49.35153383 49.64166176
50.7868806 50.09968516 51.70417798 50.15974777]
```

```
mes = np.nanmedian(n_array, axis = 1)
```

```
print("the meidan moisture for each hour is: ",mes)
```

```
the meidan moisture for each hour is: [54.355 55.805 52.04 53.735 52.69 52.82 45.65 50.52 55.63 50.34
47.79 53.92 55.04 46.26 48.25 52.72 55.645 44.76 51.43 41.81
51.41 50.79 50.5 54.29 46.2 53.66 51.655 37.08 43.3 60.78
54.45 57.96 49.62 52.69 52.53 48.82 45.29 50.79 59.73 52.945
53.78 43.015 54.22 53.1 47.37 47. 45.975 47.3 48.23 52.49
41.93 49.62 53.58 55.78 55.66 53.995 56.81 46.76 45.455 48.09
50.45 48.67 53.23 48.375 57.66 46.625 48.32 48.98 43.41 51.785
53.02 52.05 56.29 55.16 52.66 52.34 50.565 56.875 52.46 52.73
44.35 52.94 51.215 49.34 54.1 52.53 43.97 53.46 54.33 50.37
53.59 57.43 56.275 51.35 47.19 50.91 48.24 51.165 53.29 51.7
47.52 49.02 55.685 55.93 44.63 49.055 48.64 44.81 58.81 50.05
50.315 59.525 40.845 49.145 53.77 43.55 58.475 52.54 44.835 55.87
47.55 54.26 51.75 48.965 46.715 47.635 51.415 55.55 46.6 41.51
50.07 45.76 48.33 43.895 43.9 53.395 52.41 52.1 52.19 49.21
53.66 48.99 48.685 49.96 53.11 48.19 50. 42.48 46.26 51.6
48.08 51.72 52.23 47.605 50.66 46.09 53.71 54.94 49.185 54.19
52.71 52.6 62.83 57.58 51.19 44.54 51.47 48.39 50.29 57.765
40.53 51.78 55.31 59.74 52.615 52.51 45.085 48.05 47.63 47.32
45.665 50.52 45.28 52.18 48.655 51.88 56.02 51.69 56.62 47.94
49.52 48.93 51.82 55.995 54.38 44.12 51.66 48.14 44.73 53.02
45.55 45.975 54.255 57.26 52.08 47.215 61.3 53.105 50.09 42.755]
```

```
m = np.nanmax(n_array)
n = np.nanmin(n_array)
print("maximum reading is: ",m,"\n","minimum reading is: ",n)
```

[46]

```
... maximum reading is: 159.93
... minimum reading is: -19.93
```

```
x_norm[:5]
```

```
array([[0.22873346, 0.48254198, 0.37618147, 0.56983209, 0.22940064,
        nan, 0.28166352, 0.35850106, 0.32814411, 0.55726676,
        0.19737574, 0.33064606, 0.31635717, nan, 0.57322362,
        0.28372067, nan, 0.22606472, 0.53953075, 0.54420105,
        0.18831313, 0.54892694, 0.52946736, 0.44223285, 0.28010675,
        0.43784054, 0.60130101, 0.2826087 , 0.33086845, 0.34832648,
        0.31202046, 0.45196264, 0.37834983, 0.34988324, 0.59018125,
        0.36878683, 0.22995663, 0.41849216, 0.42649839, 0.23801846,
        0.50650506, 0.22573112, 0.60986323, nan, 0.32375181,
        0.24318915, 0.56988769, 0.50822862, 0.4490715 , 0.34927166,
        0.50778383, 0.28638941, 0.51067497, 0.23112421, 0.4581341 ,
        0.58951407, 0.19576337, 0.10269098, 0.41309908, 0.43255866,
        0.44184366, nan, 0.41293228, 0.51306572, 0.35433115,
        nan, 0.55654398, 0.20571556, 0.22334038, 0.96591794,
        0.40509285, 0.40325809, 0.56999889, 0.48976982, 0.47903925,
        0.58017347, 0.2548093 , 0.51084177, 0.38107417, 0.41738018,
        0.51512287, 0.27738241, 0.36172579, 0.58223062, 0.42866674,
        0.5927388 , 0.21627933, 0.47942844, 0.20649394, 0.42260647,
        0.46341599, 0.51289892, 0.25508729, 0.56215946, 0.23801846,
        0.19848771, 0.25603247, 0.44790393, 0.16924274, 0.57550317],
        [0.37345713, 0.5094518 , 0.49132659, 0.17430224, 0.21466696,
        0.47475815, 0.87746025, 0.60691649, 0.32124986, 0.52802179,
```

Q3:

```
import pandas as pd
import numpy as np
df = pd.read_csv('Titanic-Dataset.csv')
len(df['Age'])
Ncol = [i for i in df.columns]
Dcol = []
Nullcol = []
for i in df.columns:
    datatype = str(df[i].dtype)
    count = ((int(df[i].isnull().sum()))/len(df[i]))*100
    Dcol.append(datatype)
    Nullcol.append(count)

# Assuming you have successfully run the loop to generate Ncol, Dcol, and Nullcol
```

```
data_dict = {
    'Column Name': Ncol,
    'Data Type': Dcol,
    'Missing (%)': Nullcol
}
```

```
inspectreport = pd.DataFrame(data_dict)
with open('inspect_report', 'w') as f:
    f.write(str(data_dict))
Inspectreport
df['Age'] = df['Age'].fillna(df.groupby(['Pclass', 'Sex'])['Age'].transform('median'))
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df.info()
df.drop(labels='Cabin', axis=1, inplace=True)
df['FamilySize'] = df['SibSp'] + df['Parch']
df['IsAlone'] = (df['FamilySize'] == 0).astype(int)
df['Age'] = df['Age'].astype('int64')
df.to_csv('titanic_cleaned.csv', index=False, encoding='utf-8')
```

OUTPUT:

```
with open('inspect_report', 'w') as f:
    f.write(str(data_dict))
inspectreport
```

[142]

```
...
```

	Column Name	Data Type	Missing (%)
0	PassengerId	int64	0.000000
1	Survived	int64	0.000000
2	Pclass	int64	0.000000
3	Name	object	0.000000
4	Sex	object	0.000000
5	Age	float64	19.865320
6	SibSp	int64	0.000000
7	Parch	int64	0.000000
8	Ticket	object	0.000000
9	Fare	float64	0.000000
10	Cabin	object	77.104377
11	Embarked	object	0.224467

```
df.info()
```

[65]

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          891 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Cabin        204 non-null    object
11  Embarked     891 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          891 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Embarked     891 non-null    object
dtypes: float64(2), int64(5), object(4)
memory usage: 76.7+ KB
```



```
df['FamilySize'] = df['SibSp'] + df['Parch']
```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	FamilySize
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S	1
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	1
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S	1
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S	0

```
df.head(10)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	FamilySize	IsAlone
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S	0	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S	1	0
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S	0	1
5	6	0	3	Moran, Mr. James	male	25.0	0	0	330877	8.4583	Q	0	1
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	S	0	1
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	S	4	0
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	S	2	0
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	C	1	0

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    object
4   Sex          891 non-null    object
5   Age          891 non-null    int64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    object
9   Fare         891 non-null    float64
10  Embarked     891 non-null    object
11  FamilySize   891 non-null    int64
12  IsAlone      891 non-null    int64
dtypes: float64(1), int64(8), object(4)
memory usage: 90.6+ KB
```

Q4:

```
import pandas as pd
df1 = pd.read_csv('titanic_cleaned.csv')
df2 = pd.read_csv('ticket_fares.csv')
Merg_dataframe=df1.merge(df2,left_on='Ticket' , right_on='Ticket')
Merg_dataframe
Merg_dataframe['AgeGroup'] = pd.cut(Merg_dataframe['Age'], bins = [0,12,19,59,200] ,
labels=['Child', 'Teen', 'Adult' , 'Senior'])
Merg_dataframe.groupby(['Sex' , 'AgeGroup'])['Survived'].mean()
```

```
with open('report.txt', 'w') as f:
```

```
    f.write('Women show a significantly higher survival rate than men, indicating that gender played  
a major role in rescue priority.\nChildren also have a noticeably better chance of survival compared  
to adult men, though their advantage is smaller than that of women.\nWhile not every female or  
child survived—and some adult men in higher classes did—the numbers clearly lean toward  
preferential treatment for women and younger passengers during evacuation.')
```

```
#Test Hypothesis 2
```

```
Merg_dataframe.groupby('Pclass')['Survived'].mean()
```

```
Merg_dataframe['FareBin'] = pd.qcut(Merg_dataframe['Fare_x'], 4, labels=['Low', 'Medium',  
'High', 'VeryHigh'])
```

```
Merg_dataframe.groupby('FareBin')['Survived'].mean()
```

```
with open('report.txt', 'a') as f:
```

```
    f.write('\n\nThe data strongly supports the “wealth” hypothesis.\nBoth Method A and Method  
B show that passengers in higher classes or higher fare groups had better survival rates.\nEven  
though the methods group the data differently, they lead to the same conclusion: wealth increased  
the chances of survival.')
```

OUTPUT:

```
df1.head()
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked	FamilySize	IsAlone	
0	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.2500	S	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38	1	0	PC 17599	71.2833	C	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.9250	S	0	1
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	S	1	0
4	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.0500	S	0	1

```
df2.head()
```

```
df2.head()
```

	Ticket	Fare
0	A/5 21171	7.2500
1	PC 17599	71.2833
2	STON/O2. 3101282	7.9250
3	113803	53.1000
4	373450	8.0500

Merg_dataframe																Python
	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare_x	Embarked	FamilySize	IsAlone	Fare_y	AgeGroup	
0	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.2500	S	1	0	7.2500	Adult	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38	1	0	PC 17599	71.2833	C	1	0	71.2833	Adult	
2	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.9250	S	0	1	7.9250	Adult	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	S	1	0	53.1000	Adult	
				Futrelle, Mrs.												

```
Merg_dataframe.groupby(['Sex' , 'AgeGroup'])['Survived'].mean()

C:\Users\Sibghatullah\AppData\Local\Temp\ipykernel_6120\3127482570_py:1: FutureWarning: The default of observed=False is deprecated
Merg_dataframe.groupby(['Sex' , 'AgeGroup'])['Survived'].mean()

Sex    AgeGroup
female Child    0.388350
        Teen     0.753247
        Adult    0.699561
        Senior   1.000000
male    Child    0.320000
        Teen     0.086957
        Adult    0.202624
        Senior   0.111111
Name: Survived, dtype: float64
```

```
#Test Hypothesis 2
Merg_dataframe.groupby('Pclass')['Survived'].mean()

Pclass
1    0.689055
2    0.480000
3    0.239057
Name: Survived, dtype: float64
```

Merg_dataframe																Python
ssengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare_x	Embarked	FamilySize	IsAlone	Fare_y	AgeGroup	FareBin	
1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.2500	S	1	0	7.2500	Adult	Low	
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38	1	0	PC 17599	71.2833	C	1	0	71.2833	Adult	VeryHigh	
3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.9250	S	0	1	7.9250	Adult	Low	
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.1000	S	1	0	53.1000	Adult	High	

```
Merg_dataframe.groupby('FareBin')['Survived'].mean()
```

```
C:\Users\Sibghatullah\AppData\Local\Temp\ipykernel_6120\883215743.py:1: FutureWarning: The default value of infer_objects will be False in the future.  
Merg_dataframe.groupby('FareBin')['Survived'].mean()
```

```
FareBin  
Low      0.229426  
Medium   0.441687  
High     0.336449  
VeryHigh 0.609418  
Name: Survived, dtype: float64
```